Binary Search Tree and Red Black Tree

Author: Eric Zhou
Email: eric_ds_zhou@outlook.com
Date: Oct 20, 2026

Introduction

This technical note summarizes the tools and logic used in Binary Search Tree and Red Black Tree

Table of contents

Practice Problem: BST Left Rotation and Balance Update (15 Points)

Consider a Binary Search Tree (BST) in which each node `v` stores the following information:

- `v.left`: Pointer to the left child
- `v.right`: Pointer to the right child
- `v.parent`: Pointer to the parent
- `v.height`: Height of the subtree rooted at `v`
- `v.balance`: Absolute difference between the heights of its left and right subtrees

The balance of a node is defined as:

$$v.balance = |v.left.height - v.right.height|$$

The height of a node is defined as:

$$v.height = 1 + \max(v.left.height, v.right.height)$$

Assume `NIL.height = 0`.

Given Function

You are given the following function:

```
LeftRotate(T, z)
```

This function performs a standard left rotation at node `z` and correctly updates all parent and child pointers. However, it **does not update any height or balance values**.

Task

Write pseudocode for a function:

```
LeftRotateAndUpdateBalance(T, z)
```

Your function must:

1. Perform a left rotation at node `z`.
2. Correctly update the `height` and `balance` of all potentially affected nodes.

3. Avoid unnecessarily traversing the entire tree.
4. Assume `z.right != NIL`.

You may use `max(a, b)` and `abs(a)`.

Complexity Analysis

Analyze the **worst-case time complexity** of your algorithm in terms of the height h of the tree.

Solution

```
LeftRotateAndUpdateBalance(T, z) {
    LeftRotate(T, z)

    while (z != NIL) {
        z.height = 1 + max(z.left.height, z.right.height)
        z.balance = |z.left.height - z.right.height|
        z = z.parent()
    }
}
```

Worst-case Time Complexity:

\[

\boxed{\Theta(h)}

\]

After the left rotation, the algorithm updates the height and balance of each node along the path from the original `z` to the root. Each update takes constant time, and there are at most $h + 1$ nodes on this path.

Therefore, the worst-case time complexity is $\Theta(h)$.